WORKSHOP GOOGLE CLOUD APPLIED AI • LAB II (PRODUCT_ANALYTICS)

NovaHome — Analisi Intelligente delle Recensioni

Dal Testo Non Strutturato alla Causa Radice di Fabbrica con BigQuery AI

9 Moduli Didattici × 4 Pagine: Obiettivo di Business • Diagramma E/R • Listato SQL • Particolarità Tecniche

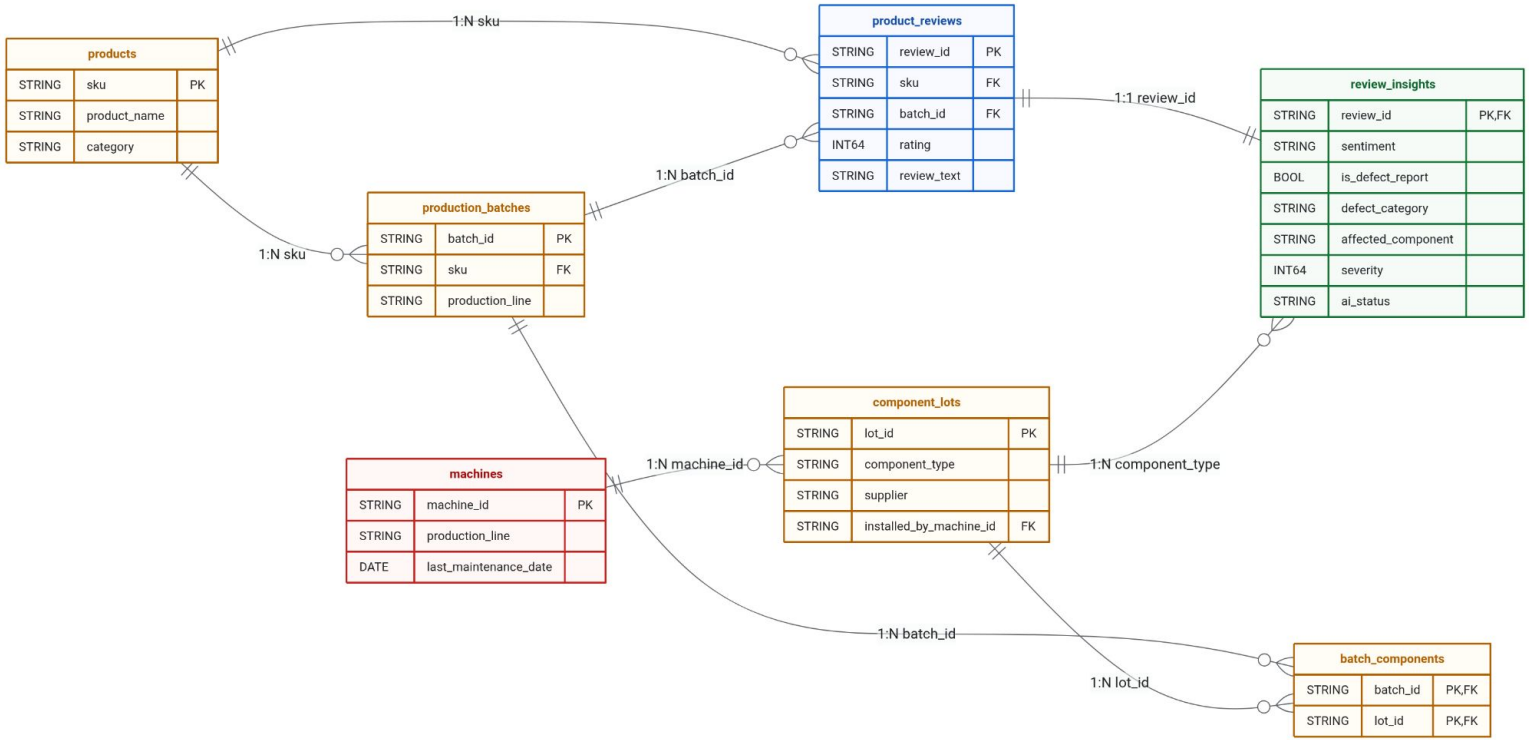
BLU — Dati Grezzi & Funzioni BigQuery AI
product_reviews • AI.IF, AI.CLASSIFY, AI.SCORE, AI.GENERATE, AI.AGG, AI.FORECAST

VERDE — Tabelle Arricchite AI & Schema Tipizzato
review_insights • output_schema • MINIMIZE_COST • TimesFM 2.5

GIALLO — Grafo Supply Chain (BOM) & Join SQL
products, batches, batch_components, component_lots • LEFT JOIN, GROUP BY

ROSSO — Causa Radice Fisica & Resilienza
machines (CAL-02) • Lotto HE-4471 (77%) • Ciclo REPEAT/UNTIL 429

Mappa E/R Completa



`sql/02_quick_triage.sql (Query 2a)`

Separare i veri difetti fisici dal rumore delle recensioni contenendo i costi operativi

IL PROBLEMA DI BUSINESS
Rumore nelle Recensioni

- Il 65% delle recensioni riguarda spedizioni, gusti personali o giudizi a 5 stelle
- Leggere e classificare manualmente ogni singolo testo non scala su migliaia di reclami
- Il team Qualità ha bisogno di una prima scrematura rapida ed economica

LE DOMANDE DI BUSINESS
Tre Segnali per Ogni Reclamo

- La recensione segnala un vero difetto fisico/sicurezza o un reclamo commerciale?
- In quale categoria di guasto rientra il problema (termico, elettrico, meccanico, estetico)?
- Qual è il livello di gravità percepito dal cliente su una scala da 1 a 5?

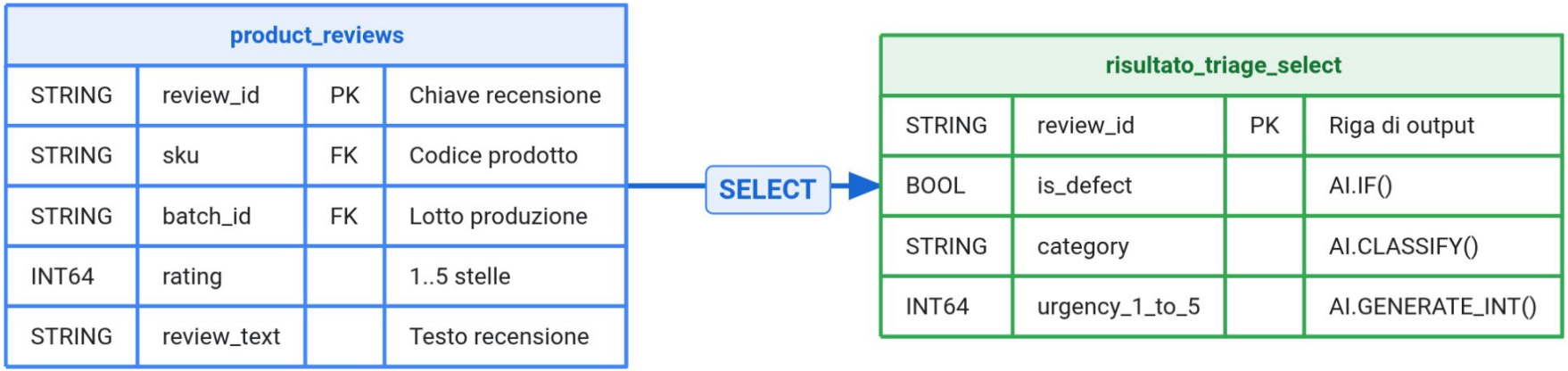
OBIETTIVO ATTESO
Triage Rapido e Sostenibile

- Isolare immediatamente i reclami reali di guasto o sicurezza dal resto del catalogo
- Classificare e pesare ogni segnalazione mantenendo minimo il costo unitario di analisi
- Fornire al team Qualità una vista esplorativa immediata sui prodotti più critici

Punto Chiave: Separare Difetti Reali e Rumore • Tassonomia a 5 Categorie • Stima Urgenza 1–5 a Costo Minimo

`sql/02_quick_triage.sql (Query 2a)`

Scansione su singola tabella di recensioni grezze con arricchimento AI al volo



sql/02_quick_triage.sql (Query 2a)

Query 2a — Triage booleano, classificazione e urgenza intera in un'unica SELECT

```
SELECT r.review_id, r.sku, r.rating,
  AI.IF(('Defect or safety issue: ', r.review_text),
    connection_id => 'us.vertex-ai-conn',
    optimization_mode => 'MINIMIZE_COST') AS is_defect,
  AI.CLASSIFY(r.review_text,
    categories => ['thermal_safety', 'electrical_failure',
      'mechanical_defect', 'cosmetic_issue', 'not_a_defect'],
    connection_id => 'us.vertex-ai-conn',
    optimization_mode => 'MINIMIZE_COST') AS category,
  AI.GENERATE_INT(('Rate urgency 1..5: ', r.review_text),
    connection_id => 'us.vertex-ai-conn',
    endpoint => 'gemini-2.5-flash').result AS urgency_1_to_5
FROM `mfg_quality_demo.product_reviews` r;
```

🔵 **AI.IF(prompt, ...) ➔ BOOL**
In: tupla (istruzione, review_text), connection_id, optimization_mode. Out: scalare BOOL diretto (is_defect).

🟢 **AI.CLASSIFY(input, categories => [...]) ➔ STRING**
In: review_text, array categories (5 stringhe), connection_id, optimization_mode. Out: scalare STRING (category).

🟡 **AI.GENERATE_INT(prompt, endpoint => ...) ➔ STRUCT**
In: tupla prompt, connection_id, endpoint. Out: STRUCT con campo .result tipizzato INT64 (urgency_1_to_5).

`sql/02_quick_triage.sql (Query 2a)`

Funzioni AI specializzate e distillazione automatica del modello riducono il TCO

FUNZIONI AI SPECIALIZZATE
Tipi SQL Forti per Ogni Task

• AI.IF restituisce direttamente un BOOL nativo (niente parsing di 'yes'/'true')

• AI.CLASSIFY vincola l'output esattamente alle 5 stringhe dell'array categories

• AI.GENERATE_INT restituisce una STRUCT con .result tipizzato INT64

OTTIMIZZAZIONE DEI COSTI
optimization_mode = MINIMIZE_COST

• Supportata sia da AI.IF sia da AI.CLASSIFY per carichi ad alto volume

• Usa un modello teacher Gemini su un campione e distilla un modello student leggero

• AI.GENERATE_INT non usa distillazione: invoca direttamente il modello Flash specificato

TRADE-OFF ARCHITETTURALE
Esplorazione vs Produzione

• Nel Modulo 03 vedremo come comprimerle in 1 sola chiamata con AI.GENERATE

Costrutto Chiave: AI.IF + AI.CLASSIFY + AI.GENERATE_INT

`sql/02_quick_triage.sql (Query 2b)`

Ordinare le recensioni negative per pericolo fisico reale anziché solo per numero di stelle

IL PROBLEMA DI BUSINESS
Tutte le review con 1 stella sembrano uguali

• Un graffio estetico e un cavo che prende fuoco possono avere entrambi 1 stella

• Il semplice voto numerico non distingue un cliente insoddisfatto da un pericolo d'incendio

• Il team Safety deve ispezionare per primi i casi di fumo, scintille o ustioni

LE DOMANDE DI BUSINESS
Misurare il Pericolo Reale

• Quanto è grave il pericolo fisico descritto nel testo libero su una scala da 0 a 100?

• Come evitare i pareggi (ex-aequo) tipici di una scala discreta 1..5 quando stabiliamo le priorità?

• Quali sono le 15 segnalazioni più pericolose dell'intero catalogo da gestire subito?

OBIETTIVO ATTESO
Coda di Priorità per il Team Safety

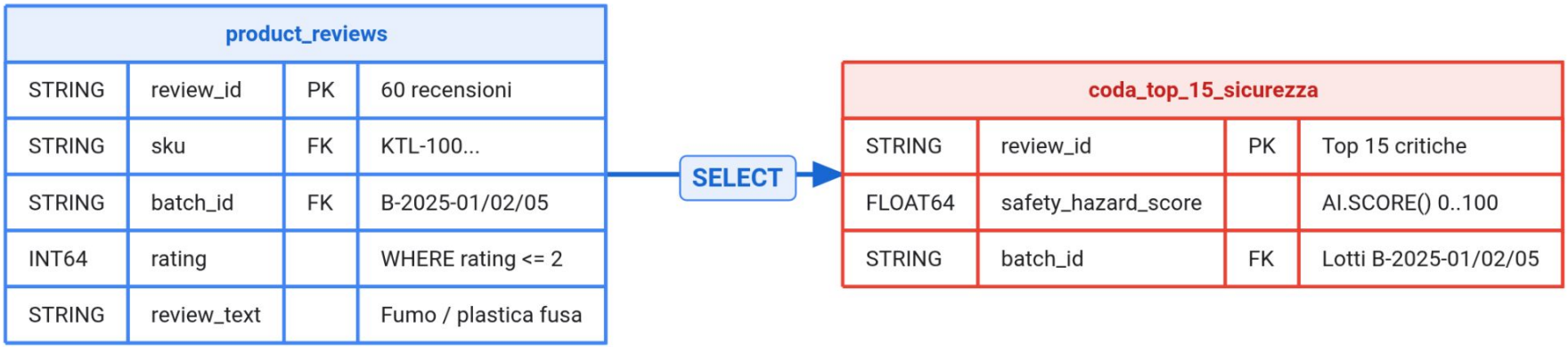
• Portare automaticamente in cima alla lista i reclami che descrivono rischi termici o elettrici gravi

• Separare le vere emergenze di sicurezza dai reclami a 1–2 stelle legati a ritardi di consegna

• Fornire una coda di ispezione immediata e ordinata per gravità

Punto Chiave: Punteggio Continuo di Rischio Fisico (0–100) • Zero Ex-Aequo in Classifica • Coda Top-15 Emergenze

Punteggio di rischio con AI.SCORE



sql/02_quick_triage.sql (Query 2b)

Query 2b — Classifica delle 15 segnalazioni a maggior rischio fisico con AI.SCORE

```sql
SELECT
  r.review_id, r.sku, r.batch_id, r.rating,
  AI.SCORE(
    ('Urgency of physical safety hazard such as fire, smoke, ',
     'melting plastic, electric shock, or burn risk: ',
     r.review_text),
    connection_id => 'us.vertex-ai-conn'
  ) AS safety_hazard_score,
  r.review_text
FROM `mfg_quality_demo.product_reviews` r
WHERE r.rating <= 2
ORDER BY safety_hazard_score DESC
LIMIT 15;
```

🔵 **AI.SCORE(prompt, connection_id => ...) ➔ FLOAT64**
In: tupla (criterio di scoring, review_text) e connection_id. Out: scalare FLOAT64 diretto (0–100, senza .result).

🟡 **WHERE r.rating <= 2**
In: predicato booleano sulla colonna INT64 rating applicato in ingresso sulla tabella product_reviews.

🔴 **ORDER BY safety_hazard_score DESC LIMIT 15**
In: alias scalare FLOAT64 calcolato nella SELECT. Out: prime 15 righe ordinate per punteggio decrescente.

`sql/02_quick_triage.sql (Query 2b)`

Combinare il filtro relazionale SQL con lo scoring continuo gestito dall'LLM

PERCHÉ NON AI.GENERATE_INT?
AI.SCORE vs AI.GENERATE_INT in ORDER BY

• Entrambe possono andare in ORDER BY, ma una scala intera 1..5 crea troppi ex-aequo

• AI.SCORE genera automaticamente una rubrica di valutazione coerente sul criterio

• Restituisce direttamente un FLOAT64 continuo (0–100) ideale per classifiche Top-K

OTTIMIZZAZIONE SQL + AI
Filtro Preventivo (WHERE <= 2)

• Il motore SQL applica prima WHERE r.rating <= 2 a costo zero

• AI.SCORE viene invocata solo sulle recensioni critiche (1 o 2 stelle)

• Combinare filtri relazionali e funzioni AI dimezza latenza e token consumati

ESPRESSIONI SQL STANDARD
Coda di Priorità Safety

• Lo scalare FLOAT64 di AI.SCORE viene ordinato con ORDER BY safety_hazard_score DESC

• Permette di creare viste di allerta per i team di Sicurezza Prodotto

• Rivela subito che i casi più gravi si concentrano sui lotti B-2025-01, 02 e 05

Costrutto Chiave: AI.SCORE in ORDER BY per Prioritizzazione del Rischio Fisico

`sql/03_async_enrichment_pipeline.sql (Query 3a)`

Costruire un patrimonio informativo strutturato e continuo dalle recensioni in arrivo

IL PROBLEMA DI BUSINESS
Analisi Ripetitive e Costose

• Rianalizzare da zero l'intero storico recensioni a ogni report moltiplica tempi e costi

• Ogni giorno arrivano nuove recensioni che devono essere integrate senza fermare i sistemi

• Eventuali picchi di traffico o rallentamenti temporanei non devono far perdere dati

LE DOMANDE DI BUSINESS
Dal Testo Libero al Dato Strutturato

• Quali informazioni chiave (sentiment, difetto, categoria, componente, gravità, sintesi) servono per ogni recensione?

• Come processare esclusivamente le recensioni nuove arrivate dall'ultimo aggiornamento?

• Come garantire che eventuali errori temporanei vengano recuperati automaticamente?

OBIETTIVO ATTESO
Scheda Tecnica per Ogni Recensione

• Trasformare ogni recensione testuale in una scheda strutturata pronta per i report aziendali

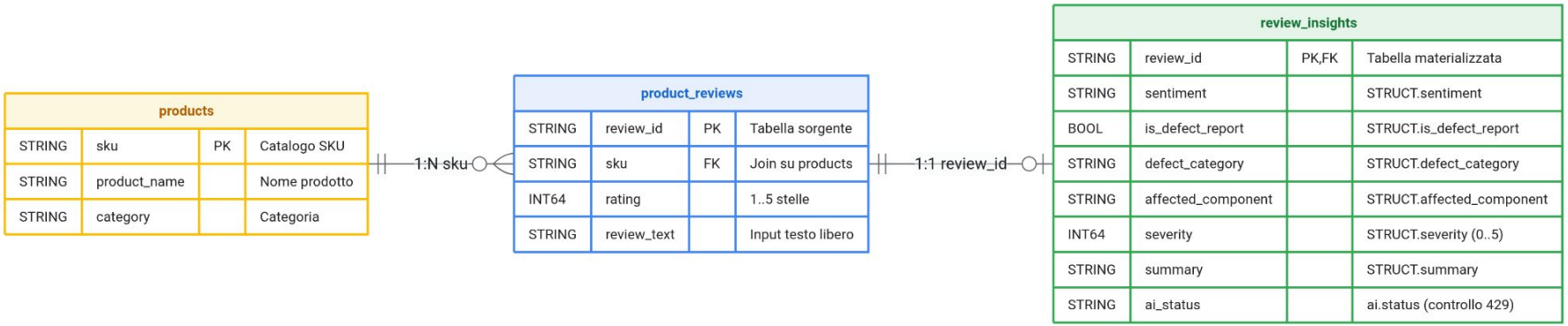
• Individuare dal racconto del cliente quale specifico componente fisico ha ceduto

• Disporre di un processo affidabile e aggiornabile continuamente sui nuovi arrivi

Punto Chiave: Estrazione Multi-Attributo Standardizzata • Elaborazione Solo dei Nuovi Arrivi • Continuità Operativa

`sql/03_async_enrichment_pipeline.sql (Query 3a)`

Left Anti-Join incrementale dalle recensioni grezze alla tabella review_insights



sql/03_async_enrichment_pipeline.sql (Query 3a)

Query 3a — Stored Procedure incrementale con output_schema e recupero errori 429

```
CREATE OR REPLACE PROCEDURE `mfg_quality_demo.enrich_new_reviews`()
BEGIN
  REPEAT
    DELETE FROM `mfg_quality_demo.review_insights`
    WHERE ai_status LIKE '%A retryable error occurred%';
    INSERT INTO `mfg_quality_demo.review_insights`
    SELECT review_id, ai.sentiment, ai.is_defect_report,
      ai.defect_category, ai.affected_component,
      ai.severity, ai.summary, ai.status, CURRENT_TIMESTAMP()
    FROM (
      SELECT r.review_id, AI.GENERATE(
        ('Analyze ', p.product_name, ': ', r.review_text),
        connection_id => 'us.vertex-ai-conn',
        endpoint => 'gemini-2.5-flash',
        output_schema => 'sentiment STRING, is_defect_report BOOL,
          defect_category STRING, affected_component STRING,
          severity INT64, summary STRING') AS ai
      FROM `mfg_quality_demo.product_reviews` r
      JOIN `mfg_quality_demo.products` p USING (sku)
      LEFT JOIN `mfg_quality_demo.review_insights` i USING (review_id)
      WHERE i.review_id IS NULL);
  UNTIL (SELECT COUNT(*) FROM `mfg_quality_demo.review_insights`
         WHERE ai_status LIKE '%retryable error%') = 0 END REPEAT;
END;
```

🟢 **AI.GENERATE(..., output_schema => '...') ➔ STRUCT**
In: tupla prompt, connection_id, endpoint, output_schema DDL. Out: STRUCT ai con 6 campi tipizzati + ai.status (STRING).

🟡 **LEFT JOIN ... USING (review_id) WHERE i.review_id IS NULL**
In: chiave review_id tra product_reviews (r) e review_insights (i). Out: sole righe di r con i.review_id NULL.

🔴 **REPEAT ... DELETE ... INSERT ... UNTIL count = 0**
In: blocco procedurale SQL con DELETE su ai_status LIKE '%retryable error%' e condizione scalare booleana UNTIL.

`sql/03_async_enrichment_pipeline.sql (Query 3a)`

Tre pattern di ingegneria di produzione in un'unica Stored Procedure BigQuery

1. OUTPUT STRUTTURATO
output_schema Multi-Campo

• Un'unica chiamata AI.GENERATE restituisce una STRUCT SQL con 6 campi tipizzati

• Più veloce ed economico rispetto a 6 funzioni AI separate sulla stessa riga

• Vincola affected_component al vocabolario esatto della distinta base (BOM)

2. ANTI-JOIN INCREMENTALE
LEFT JOIN ... WHERE IS NULL

• Confronta product_reviews con review_insights sulla chiave primaria review_id

• Invia a Vertex AI esclusivamente le recensioni non ancora presenti a sistema

• Rende la procedura enrich_new_reviews() idempotente e sicura da schedulare

3. RESILIENZA ALLE QUOTE
Ciclo REPEAT + Filtro ai_status

• AI.GENERATE non fa fallire l'intera query se una riga riceve HTTP 429: scrive l'errore in .status

• Il ciclo REPEAT elimina le righe con ai_status LIKE '%retryable error%'

• L'anti-join successiva riprocessa automaticamente solo le righe fallite

Costrutto Chiave: AI.GENERATE con Output Strutturato + Anti-Join Incrementale + Auto-Retry Quota

`sql/04_root_cause_analysis.sql (Query 4a + 4b + 4c)`

Risalire dai sintomi descritti dai clienti al lotto di componenti e al macchinario responsabile

IL PROBLEMA DI BUSINESS
Il Cliente Non Conosce la Fabbrica

- Il cliente descrive il sintomo ('non si spegne mai'), ma non sa quale componente interno sia guasto
- Ogni lotto di prodotto finito assembla 5 componenti diversi provenienti da più fornitori
- Un richiamo generalizzato dell'intero catalogo costerebbe milioni e danneggerebbe il brand

LE DOMANDE DI BUSINESS
Fornitore o Macchinario di Linea?

- Quale specifico lotto di componenti registra la percentuale più alta di reclami sul campo?
- Il difetto dipende dalla materia prima del fornitore o dal macchinario che lo ha installato?
- Altri lotti dello stesso fornitore montati su linee diverse presentano lo stesso problema?

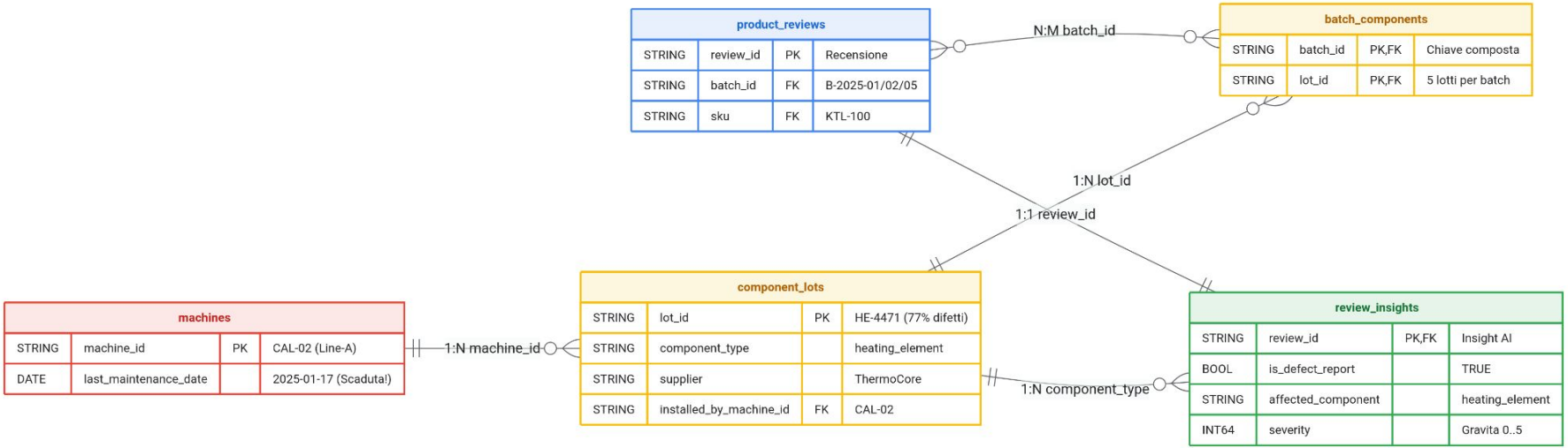
OBIETTIVO ATTESO
Richiamo Chirurgico e Verifica

- Isolare con precisione il singolo lotto difettoso e i soli lotti di produzione impattati
- Accertare con dati oggettivi se la responsabilità è del fornitore o di una macchina di stabilimento
- Intervenire solo sulla linea e sui lotti coinvolti risparmiando milioni in richiami inutili

Punto Chiave: Tracciabilità Sintomo ➔ Componente ➔ Lotto ➔ Macchina • Verifica Responsabilità Fornitore vs Fabbrica

`sql/04_root_cause_analysis.sql (Query 4a + 4b + 4c)`

Attraversamento di 5 tabelle della supply chain guidato da affected_component = component_type



sql/04_root_cause_analysis.sql
(Query 4a + 4b + 4c)

Query 4a & 4b — Vista difettosità per lotto e risalita alla calibratrice CAL-02

```
CREATE OR REPLACE VIEW `mfg_quality_demo.v_component_lot_defect_rates`
AS
SELECT cl.lot_id, cl.component_type, cl.supplier,
cl.installed_by_machine_id,
  COUNT(DISTINCT r.review_id) AS total_reviews_for_lot,
  COUNT(DISTINCT CASE WHEN i.is_defect_report
    AND i.affected_component = cl.component_type
    THEN r.review_id END) AS defect_reviews,
  ROUND(SAFE_DIVIDE(COUNT(DISTINCT CASE WHEN i.is_defect_report
    AND i.affected_component = cl.component_type THEN r.review_id
END),
    COUNT(DISTINCT r.review_id)), 2) AS implication_rate
FROM `mfg_quality_demo.component_lots` cl
JOIN `mfg_quality_demo.batch_components` bc USING (lot_id)
JOIN `mfg_quality_demo.product_reviews` r   USING (batch_id)
JOIN `mfg_quality_demo.review_insights` i   USING (review_id)
GROUP BY 1, 2, 3, 4;
-- 4b. Roll-up per Macchina di Fabbrica:
SELECT m.machine_id, m.last_maintenance_date,
  SUM(v.defect_reviews) AS defects, ROUND(AVG(v.implication_rate), 2)
AS rate
FROM `mfg_quality_demo.v_component_lot_defect_rates` v
JOIN `mfg_quality_demo.machines` m ON m.machine_id =
v.installed_by_machine_id
GROUP BY 1, 2 ORDER BY defects DESC;
```

🟢 **COUNT(DISTINCT CASE WHEN ... THEN r.review_id END)**
In: predicato i.is_defect_report AND i.affected_component = cl.component_type. Out: conteggio INT64 (defect_reviews).

🟡 **SAFE_DIVIDE(numeratore, denominatore) ➔ FLOAT64**
In: defect_reviews (INT64) e total_reviews_for_lot (INT64). Out: rapporto FLOAT64 arrotondato con ROUND(..., 2).

🔴 **JOIN ... USING (lot_id, batch_id, review_id)**
In: chiavi di join tra le 4 tabelle e GROUP BY 1, 2, 3, 4. Out: vista per lot_id e query di roll-up su machine_id.

`sql/04_root_cause_analysis.sql (Query 4a + 4b + 4c)`

Perché il predicato ibrido semantico-relazionale è il fulcro dell'intera architettura

IL PREDICATO CHIAVE
Disambiguare la Distinta Base M:N

• Un batch (es. B-2025-01) contiene 5 componenti: resistenza, sensore, coperchio, guarnizione, cavo

• La JOIN tradizionale su batch_id assocerebbe il reclamo termico anche al coperchio!

• Il filtro i.affected_component = cl.component_type isola solo il componente citato dal cliente

TASSO DI IMPLICAZIONE
COUNT(DISTINCT CASE WHEN ...)

• Denominatore: tutte le recensioni dei batch che montano quel lot_id

• Numeratore: solo le recensioni difettose in cui l'AI ha indicato quel component_type

• HE-4471 raggiunge 17 / 22 = 77.3% con severità media 4.5 / 5.0

VERIFICA INCROCIATA
Scagionare il Fornitore ThermoCore

• Query 4b aggrega su machines: CAL-02 emerge con manutenzione ferma al 2025-01-17

• Query 4c confronta tutti i heating_element: HE-4472 (stesso fornitore ThermoCore ma su CAL-01) è al 9%

• Dimostra matematicamente che il problema è la taratura di CAL-02, non la materia prima

Costrutto Chiave: Join Relazionale Ibrida AI-Distinta Base: da Testo Libero a Lotto HE-4471 e Macchina CAL-02

`sql/05_executive_summary.sql (Query 5)`

Sintetizzare le evidenze numeriche di fabbrica in un piano d'azione esecutivo per la direzione

IL PROBLEMA DI BUSINESS
Dai Dati Tecnici alla Decisione

• La direzione operativa (VP of Manufacturing) non ha tempo di interpretare tabelle e metriche grezze

• Un riassunto generico scritto senza basi numeriche certificate rischia di contenere imprecisioni

• In una crisi di qualità serve un documento decisionale immediato, sintetico e verificabile

LE DOMANDE DI BUSINESS
Cosa Deve Decidere il Board Oggi?

• Qual è la causa radice accertata e quali evidenze numeriche giustificano tale conclusione?

• Quali azioni immediate vanno intraprese in fabbrica, in magazzino e verso i fornitori?

• Qual è l'impatto stimato sui clienti finali e quali lotti vanno messi in quarantena?

OBIETTIVO ATTESO
Briefing Esecutivo Certificato

• Produrre una nota direzionale di 250 parole ancorata al 100% alle metriche reali di fabbrica

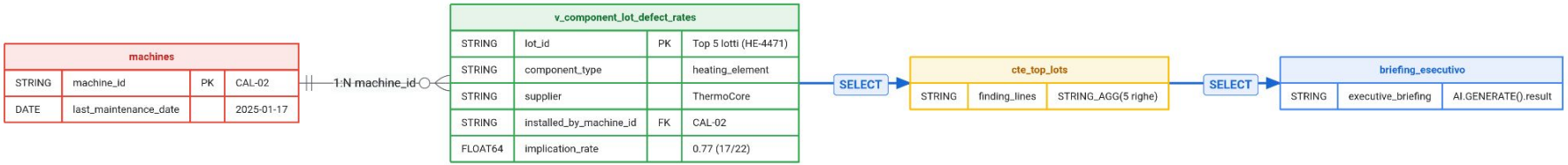
• Articolare chiaramente Causa Radice, Azioni Immediate e Impatto Clienti

• Azzerare il rischio di cifre inventate o contraddittorie nel passaggio tra analisti e management

Punto Chiave: Comunicazione Direzionale in 3 Punti • 100% Ancorata alle Metriche Certificate • Pronta per il Board

`sql/05_executive_summary.sql (Query 5)`

Compressione delle viste multi-riga in una CTE a riga singola tramite STRING_AGG



sql/05_executive_summary.sql (Query 5)

Query 5 — Pre-aggregazione deterministica in CTE e generazione del Piano d'Azione

```
WITH top_lots AS (
  SELECT STRING_AGG(FORMAT(
    'Lot %s (%s, %s, machine=%s, maint=%t): %d/%d (%.0f%%)',
    v.lot_id, v.component_type, v.supplier,
    v.installed_by_machine_id, m.last_maintenance_date,
    v.defect_reviews, v.total_reviews_for_lot,
    v.implication_rate * 100
  ), '\n' ORDER BY v.defect_reviews DESC) AS finding_lines
  FROM (
    SELECT * FROM `mfg_quality_demo.v_component_lot_defect_rates`
    ORDER BY defect_reviews DESC LIMIT 5
  ) v JOIN `mfg_quality_demo.machines` m
    ON m.machine_id = v.installed_by_machine_id
)
SELECT AI.GENERATE(
  ('Write a 250-word executive briefing with: (1) Root Cause, ',
   '(2) Immediate Actions, (3) Customer Impact.\n', t.finding_lines),
  connection_id => 'us.vertex-ai-conn',
  endpoint => 'gemini-2.5-flash').result AS executive_briefing
FROM top_lots t;
```

🟡 **FORMAT('Lot %s ...', ...) ➔ STRING**
In: stringa di formato (%s, %t, %d, %.0f%%) e 8 colonne tipizzate. Out: 1 valore STRING formattato per riga.

🔴 **STRING_AGG(..., '\n' ORDER BY ...) ➔ STRING**
In: espressione STRING per riga e separatore '\n'. Out: 1 sola riga con colonna finding_lines (STRING) nella CTE top_lots.

🔵 **AI.GENERATE((prompt, t.finding_lines), ...).result**
In: tupla (istruzione, finding_lines), connection_id, endpoint. Out: STRUCT con campo .result STRING (executive_briefing).

`sql/05_executive_summary.sql (Query 5)`

Il pattern anti-allucinazione 'SQL-to-Prompt' per il reporting direzionale

SEPARAZIONE DEI COMPITI
SQL per la Matematica, LLM per la Sintesi

• Gli LLM non devono mai calcolare aggregazioni o percentuali su decine di righe

• Il motore SQL di BigQuery calcola in modo deterministico tassi, conteggi e date

• L'LLM riceve solo l'evidenza già aggregata e certificata

COMPRESSIONE DEL CONTESTO
FORMAT + STRING_AGG(..., '\n')

• FORMAT costruisce una riga testuale strutturata per ciascuno dei 5 lotti principali

• STRING_AGG collassa le 5 righe in un unico scalare di testo (finding_lines)

• Riduce l'input della query finale a 1 sola riga = 1 sola invocazione Gemini

PROMPT STRUTTURATO
Output Azionabile in 3 Sezioni

• Il prompt richiede esplicitamente: (1) Causa Radice, (2) Azioni Immediate, (3) Impatto Clienti

• Cita i numeri esatti (77% vs 9%) per giustificare lo scagionamento del fornitore

• Pronto per essere inviato via email o mostrato in una dashboard Looker

Costrutto Chiave: Pre-Aggregazione Deterministica SQL + Sintesi AI.GENERATE in Singola Chiamata

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7a)`

Individuare automaticamente quale combinazione di fattori spiega l'anomalia di qualità

IL PROBLEMA DI BUSINESS
E se Avessimo 50 Dimensioni?

• In uno stabilimento reale esistono decine di variabili (turno, linea, fornitore, lotto, macchina, SKU...)

• Verificare manualmente ogni singola ipotesi o incrocio richiede giorni di analisi per tentativi

• Le cause radice più insidiose nascono spesso dall'interazione tra più fattori contemporaneamente

LE DOMANDE DI BUSINESS
Quali Fattori Spingono i Difetti?

• Quale attributo singolo o combinazione di attributi concentra un numero anomalo di difetti?

• Quanto è più frequente il guasto in quel segmento rispetto alla media normale di fabbrica?

• Quali combinazioni hanno il maggiore impatto complessivo sul totale dei reclami?

OBIETTIVO ATTESO
Diagnosi Multi-Fattore Istantanea

• Esplorare in automatico tutte le combinazioni possibili della catena produttiva senza ipotesi a priori

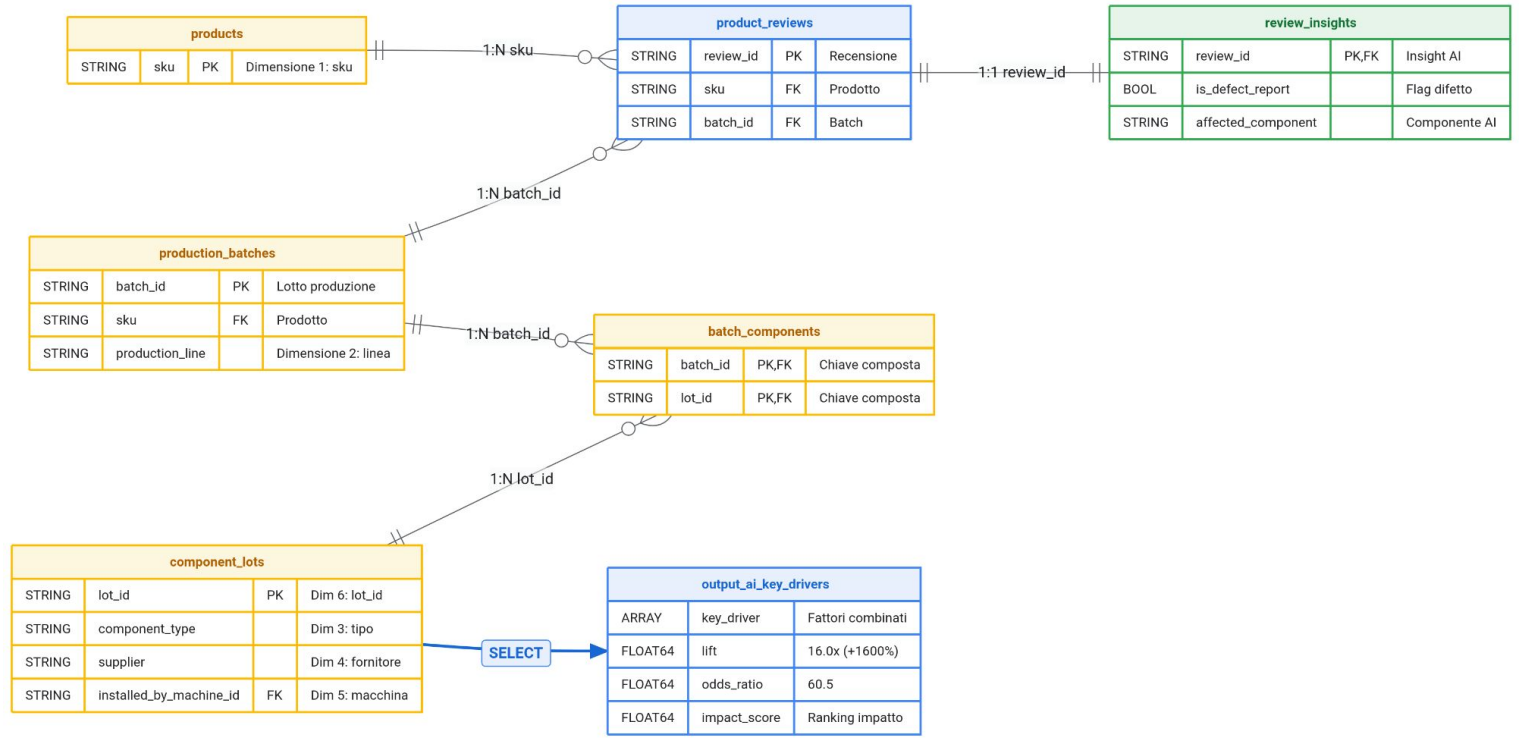
• Ordinare i fattori sospetti per rilevanza statistica e volume di difetti spiegati

• Ridurre da giorni a pochi secondi il tempo necessario per isolare un'anomalia complessa

Punto Chiave: Esplorazione Automatica Multi-Dimensionale • Zero Ipotesi Manuali a Priori • Ranking per Impatto Reale

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7a)`

Join denormalizzata a 6 tabelle passata in input alla funzione tabellare AI.KEY_DRIVERS



sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7a)

Query 7a — Scoperta automatica dei fattori di difettosità con AI.KEY_DRIVERS

```
SELECT key_driver, true_count AS defect_matches, total_count,
  ROUND(lift, 2) AS lift, ROUND(odds_ratio, 2) AS odds_ratio,
  ROUND(unexpected_True_ratio, 2) AS unexpected_defect_ratio,
  ROUND(impact_score, 2) AS impact_score
FROM AI.KEY_DRIVERS(
  TABLE(
    SELECT p.sku, pb.production_line, cl.component_type,
      cl.supplier, cl.installed_by_machine_id, cl.lot_id,
      (i.is_defect_report AND
       i.affected_component = cl.component_type) AS
is_implicated_defect
    FROM `mfg_quality_demo.review_insights` i
    JOIN `mfg_quality_demo.product_reviews` r     USING (review_id)
    JOIN `mfg_quality_demo.products` p            USING (sku)
    JOIN `mfg_quality_demo.production_batches` pb USING (batch_id)
    JOIN `mfg_quality_demo.batch_components` bc   USING (batch_id)
    JOIN `mfg_quality_demo.component_lots` cl     USING (lot_id)
  ), is_implicated_defect
) ORDER BY impact_score DESC LIMIT 10;
```

🔵 **FROM AI.KEY_DRIVERS(TABLE(SELECT ...), target)**
In (Arg 1): TABLE(...) contenente 6 colonne dimensione (STRING) e 1 colonna target (BOOL). Sintassi TVF in FROM.

🟢 **Arg 2: is_implicated_defect (BOOL)**
In (Arg 2): identificatore della colonna booleana nella TABLE di input rispetto a cui calcolare le metriche.

🔴 **Colonne di Output della TVF**
Out: key_driver (ARRAY<STRING>), true_count (INT64), total_count (INT64), lift, odds_ratio, unexpected_True_ratio, impact_score (FLOAT64).

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7a)`

Come interpretare le metriche statistiche di contributo restituite da AI.KEY_DRIVERS

COME FUNZIONA AI.KEY_DRIVERS
Funzione Tabellare (TVF)

• Prende in input una TABLE(SELECT ...) con le dimensioni candidate e un target booleano

• Valuta sia i singoli attributi (es. sku='KTL-100') sia le intersezioni multi-attributo

• Filtra il rumore statistico e restituisce solo i fattori significativi

LE METRICHE STATISTICHE
Lift, Odds Ratio e Impatto

• lift: quanto è più probabile il difetto in quel segmento rispetto alla media globale (16x)

• unexpected_True_ratio: quota di difetti 'in eccesso' spiegata da quel segmento (+69%)

• impact_score: combina volume di difetti e lift per dare una priorità unica

CONFRONTO DIDATTICO
Modulo 04 vs Modulo 06

• Modulo 04 (SQL classico): ottimo quando conosci già la gerarchia da verificare

• Modulo 06 (AI.KEY_DRIVERS): imbattibile per scoprire cause radice al buio su decine di colonne

• Entrambi convergono esattamente su HE-4471 + ThermoCore + CAL-02

Costrutto Chiave: AI.KEY_DRIVERS per l'Analisi Automatica Multi-Dimensionale dei Fattori Critici

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7b)`

Fornire a ogni Product Manager un quadro sintetico di KPI e sintomi ricorrenti per prodotto

IL PROBLEMA DI BUSINESS
Un Report per Ogni Product Manager

• Ogni Product Manager è responsabile di una specifica linea di prodotto (Bollitore, Frullatore, Friggitrice...)

• I numeri da soli (# reclami, gravità media) dicono quanto è grande il problema, ma non cosa succede

• Leggere centinaia di recensioni per capire i sintomi ricorrenti di ogni prodotto richiede troppo tempo

LE DOMANDE DI BUSINESS
Numeri e Sintesi nella Stessa Vista

• Quante segnalazioni di difetto e quale gravità media registra ciascun prodotto a catalogo?

• Quali sono i sintomi ricorrenti e i lotti più citati dai clienti per quello specifico prodotto?

• Quale verifica ingegneristica prioritaria emerge dai reclami di ciascuna linea?

OBIETTIVO ATTESO
Cruscotto Qualità per Prodotto

• Affiancare in un'unica tabella gli indicatori quantitativi e il riassunto qualitativo dei guasti per prodotto

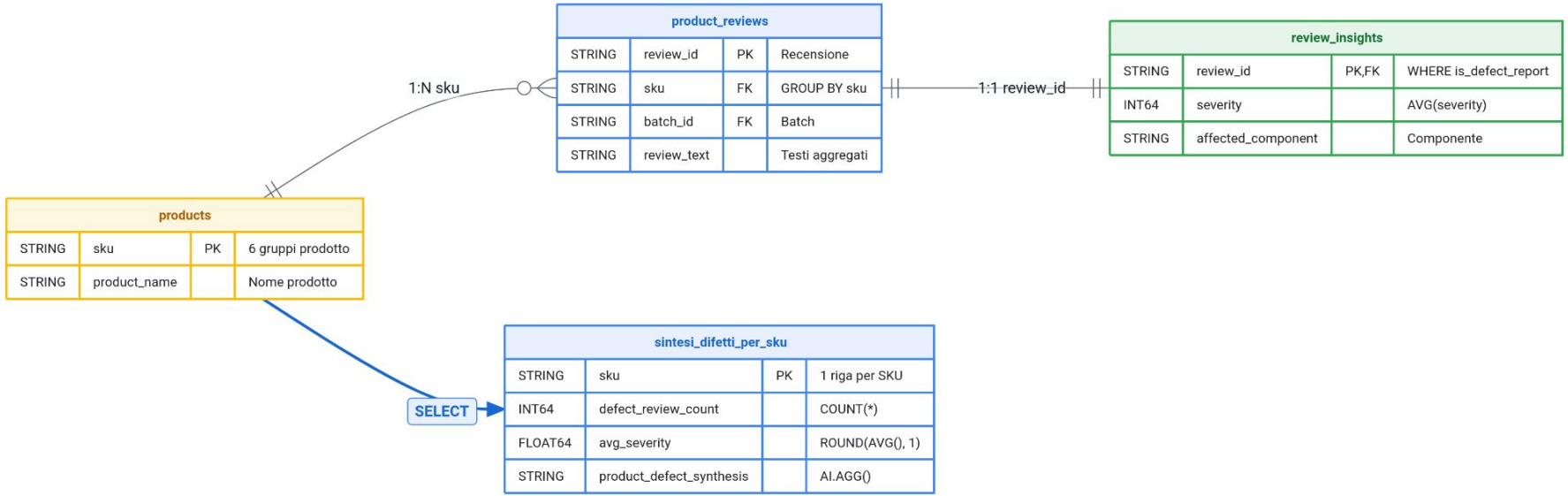
• Distinguere subito i prodotti con veri problemi strutturali da quelli con sole segnalazioni estetiche

• Offrire a ciascun responsabile di prodotto indicazioni operative immediate

Punto Chiave: Vista Unificata per Prodotto (SKU) • KPI Quantitativi + Sintesi Qualitativa dei Sintomi • Priorità per i PM

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7b)`

Aggregazione semantica molti-a-uno sulle recensioni raggruppate per products.sku



sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7b)

Query 7b — Sintesi multi-riga per prodotto usando AI.AGG dentro GROUP BY

```
SELECT
  p.sku, p.product_name,
  COUNT(*) AS defect_review_count,
  ROUND(AVG(i.severity), 1) AS avg_severity,
  AI.AGG(
    ('Batch ', r.batch_id, ' [', i.affected_component, ']: ',
     r.review_text),
    instructions => 'Summarize recurring defect symptoms, '
      || 'affected batches, and engineering check in 2 bullets.',
    connection_id => 'us.vertex-ai-conn',
    endpoint => 'gemini-2.5-flash'
  ) AS product_defect_synthesis
FROM `mfg_quality_demo.review_insights` i
JOIN `mfg_quality_demo.product_reviews` r USING (review_id)
JOIN `mfg_quality_demo.products` p        USING (sku)
WHERE i.is_defect_report = TRUE
GROUP BY p.sku, p.product_name
ORDER BY defect_review_count DESC;
```

🔵 **AI.AGG(valore_riga, ...) ➔ STRING**
In (Arg 1): tupla di colonne per riga (batch_id, affected_component, review_text). Out: scalare STRING diretto (senza .result).

🟢 **instructions => '...'**
In (Parametri nominali): instructions (STRING), connection_id (STRING), endpoint (STRING).

🟡 **GROUP BY p.sku, p.product_name**
In: chiavi di raggruppamento SQL. Out: 1 riga per SKU con COUNT(*) (INT64), AVG(severity) (FLOAT64) e AI.AGG (STRING).

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7b)`

AI.GENERATE scalare vs AI.AGG aggregata: quando usare l'una o l'altra

FUNZIONE AGGREGATA NATIVA
AI.AGG vs AI.GENERATE

• AI.GENERATE è una funzione scalare (1 riga in input ➔ 1 riga in output)

• AI.AGG è una funzione di aggregazione (N righe di un gruppo ➔ 1 valore per gruppo)

• Si scrive direttamente nella SELECT insieme a COUNT(*), AVG(), MAX() con GROUP BY

SCALABILITÀ GERARCHICA
Gestione Automatica del Contesto

• Con STRING_AGG manuale, se un gruppo ha 50.000 recensioni la stringa esplode

• AI.AGG gestisce nativamente l'aggregazione di grandi insiemi di righe per gruppo

• Accetta una tupla con i campi di ogni riga (es. batch_id, affected_component, review_text)

SEPARAZIONE ISTRUZIONE / DATI
Parametro instructions => ...

• Il primo argomento di AI.AGG contiene solo l'espressione riga-per-riga da aggregare

• Il parametro nominale instructions => '...' definisce il task di sintesi a livello di gruppo

• Evita di ripetere il testo del prompt per ogni singola riga del gruppo!

Costrutto Chiave: AI.AGG dentro GROUP BY per la Sintesi Automatica per Prodotto (SKU)

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7c + 7d)`

Collegare i bollettini tecnici dell'ingegneria con i reclami dei clienti scritti in linguaggio comune

IL PROBLEMA DI BUSINESS
Due Linguaggi che Non Si Parlano

• Il team Engineering documenta i difetti noti in bollettini tecnici con terminologia specialistica

• I clienti descrivono gli stessi guasti usando parole quotidiane basate su ciò che vedono o sentono

• Non esiste alcun codice o riferimento comune che colleghi una recensione cliente a un bollettino tecnico

LE DOMANDE DI BUSINESS
Collegare Sintomo Cliente e Nota Tecnica

• Quante recensioni dei clienti corrispondono esattamente alla modalità di guasto descritta in un bollettino?

• Quali altre segnalazioni nel catalogo descrivono un sintomo simile a un caso critico appena scoperto?

• Quando serve verificare un nesso logico causa-effetto e quando basta cercare frasi simili?

OBIETTIVO ATTESO
Ponte tra Assistenza Clienti e R&D

• Misurare subito l'impatto sul campo di ciascun bollettino tecnico contando i reclami corrispondenti

• Individuare rapidamente tutti i casi analoghi a una segnalazione critica descritta a parole

• Unificare il patrimonio informativo dell'ingegneria con la voce del cliente senza codifiche manuali

Punto Chiave: Ponte tra Lessico Tecnico e Linguaggio Cliente • Impatto sul Campo dei Bollettini • Ricerca Casi Simili

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7c + 7d)`

Join semantica logica ON AI.IF(...) e ricerca vettoriale continua con AI.SIMILARITY



sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7c + 7d)

Query 7c & 7d — Semantic JOIN con AI.IF e ricerca vettoriale con AI.SIMILARITY

```sql
-- 7c. Semantic JOIN tra Bollettini Tecnici (b) e Recensioni (r):
SELECT b.bulletin_id, COUNT(r.review_id) AS matching_reviews
FROM engineering_bulletins b
JOIN `mfg_quality_demo.product_reviews` r
  ON b.target_sku = r.sku AND r.rating <= 2
 AND AI.IF(('Review: ', r.review_text, ' matches bulletin: ',
            b.technical_description),
   connection_id => 'us.vertex-ai-conn',
   optimization_mode => 'MINIMIZE_COST') GROUP BY 1;
-- 7d. Ricerca Semantica Continua con Embeddings:
SELECT r.review_id, r.sku, r.batch_id,
  ROUND(AI.SIMILARITY(
    'kettle never stops boiling and smells like burnt plastic',
    r.review_text, connection_id => 'us.vertex-ai-conn',
    endpoint => 'text-embedding-005'), 3) AS semantic_similarity
FROM `mfg_quality_demo.product_reviews` r
ORDER BY semantic_similarity DESC LIMIT 10;
```

🔵 **JOIN ... ON ... AND AI.IF(prompt, ...) ➔ BOOL**
In (7c): tupla (r.review_text, b.technical_description), connection_id, optimization_mode. Out: BOOL nella clausola ON.

🟢 **AI.SIMILARITY(content1, content2, ...) ➔ FLOAT64**
In (7d): 2 argomenti posizionali STRING (testo cercato, r.review_text), connection_id, endpoint => 'text-embedding-005'.

🟡 **Output 7c (INT64) e 7d (FLOAT64)**
Out (7c): COUNT(r.review_id) (INT64) per bulletin_id. Out (7d): scalare FLOAT64 diretto (-1.0..1.0) in ORDER BY.

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7c + 7d)`

Scegliere tra verifica logica LLM (AI.IF) ed embedding vettoriali (AI.SIMILARITY)

QUERY 7c: JOIN ON AI.IF
Join Relazionale + Semantica

• Usa AI.IF direttamente dentro la clausola ON di una INNER JOIN

• L'LLM ragiona sul rapporto causa-effetto (es. 'deriva a 108°C' = 'non si spegne quando bolle')

• Fondamentale: aggiungere sempre un pre-filtro equijoin (b.target_sku = r.sku) per evitare O(N×M)

QUERY 7d: AI.SIMILARITY
Embedding Istantanei (text-embedding-005)

• Calcola al volo gli embedding dei due testi e restituisce la similarità coseno (-1..1)

• Non richiede di creare una tabella di embedding né di gestire VECTOR_SEARCH

• Molto più veloce ed economico di un LLM generativo per ricerche di vicinanza semantica

QUANDO USARE QUALE?
Regola Pratica per gli Studenti

• Usa AI.SIMILARITY per trovare testi 'simili' o duplicati e ordinarli per punteggio

• Usa JOIN ON AI.IF quando serve un criterio logico complesso ('A è causato da B')

• Con optimization_mode => 'MINIMIZE_COST', AI.IF diventa sostenibile anche su join ampie

Costrutto Chiave: JOIN ON AI.IF (Correlazione Logica) & AI.SIMILARITY (Ricerca Vettoriale)

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7e + 7f + 7g)`

Rilevare tempestivamente i picchi anomali di difetti e prevedere il carico futuro di reclami

IL PROBLEMA DI BUSINESS
Allerta Precoce & Pianificazione Scorte

• Un'impennata improvvisa di reclami dopo una spedizione deve far scattare subito un'allerta qualità

• Il team Ricambi e il Customer Care devono sapere in anticipo quanti reclami arriveranno la prossima settimana

• I team operativi non hanno il tempo di progettare e mantenere modelli statistici dedicati per ogni metrica

LE DOMANDE DI BUSINESS
Anomalie, Previsioni e Affidabilità

• In quali giornate il numero di difetti segnalati ha superato in modo anomalo l'andamento storico normale?

• Quanti difetti e di quale gravità media dobbiamo aspettarci nei prossimi 7 giorni?

• Quanto sono affidabili e accurate queste previsioni se confrontate con i dati reali osservati?

OBIETTIVO ATTESO
Monitoraggio Proattivo della Qualità

• Identificare automaticamente i giorni di picco anomalo non appena i difetti deviano dal trend atteso

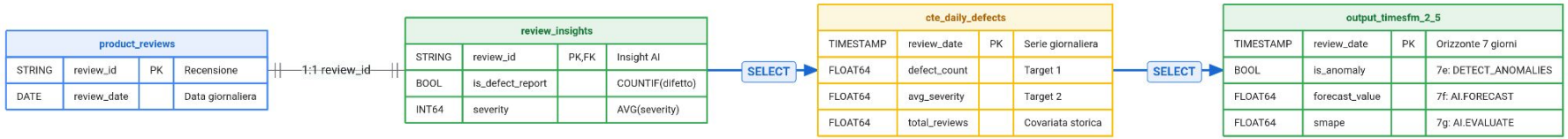
• Prevedere congiuntamente volume dei difetti e gravità media sulla settimana successiva

• Misurare oggettivamente il margine di errore previsionale per pianificare scorte e assistenza

Punto Chiave: Rilevamento Automatico dei Picchi Anomali • Previsione Congiunta a 7 Giorni • Misura dell'Accuratezza

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7e + 7f + 7g)`

Aggregazione giornaliera dei difetti in una serie storica per TimesFM 2.5



sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7e + 7f + 7g)

Query 7e, 7f & 7g — Rilevamento anomalie, previsione multivariata e backtesting TimesFM

```sql
-- Condivisa: WITH daily_defects AS (SELECT TIMESTAMP(review_date)...
--   COUNTIF(is_defect_report) AS defect_count, AVG(severity)...)
-- 7e. Rilevamento Anomalie Zero-Shot:
SELECT * FROM AI.DETECT_ANOMALIES(
  TABLE(SELECT * FROM daily_defects WHERE review_date <
'2025-03-10'),
  TABLE(SELECT * FROM daily_defects WHERE review_date >=
'2025-03-10'),
  data_col => 'defect_count', timestamp_col => 'review_date',
  model => 'TimesFM 2.5', anomaly_prob_threshold => 0.80);
-- 7f. Previsione Multivariata con Covariate Storiche:
SELECT * FROM AI.FORECAST(TABLE(daily_defects),
  timestamp_col => 'review_date',
  target_cols => ['defect_count', 'avg_severity'],
  past_covariate_cols => ['total_reviews'],
  model => 'TimesFM 2.5', horizon => 7, confidence_level => 0.90);
-- 7g. Valutazione Backtesting Zero-Shot (MAE, RMSE, sMAPE):
SELECT * FROM AI.EVALUATE(
  TABLE(SELECT * FROM daily_defects WHERE review_date <
'2025-03-10'),
  TABLE(SELECT * FROM daily_defects WHERE review_date >=
'2025-03-10'),
  data_col => 'defect_count', timestamp_col => 'review_date',
  model => 'TimesFM 2.5');
```

🔴 **AI.DETECT_ANOMALIES(TABLE(history), TABLE(target), ...)**
In (7e): 2 TABLE(...), data_col, timestamp_col, model, anomaly_prob_threshold (FLOAT64). Out: tabella con is_anomaly (BOOL).

🔵 **AI.FORECAST(TABLE(...), target_cols => [...], ...)**
In (7f): TABLE, timestamp_col, target_cols (ARRAY<STRING>), past_covariate_cols (ARRAY<STRING>), horizon (INT64), confidence_level.

🟢 **AI.EVALUATE(TABLE(history), TABLE(actuals), ...)**
In (7g): 2 TABLE(...), data_col, timestamp_col, model => 'TimesFM 2.5'. Out: tabella con MAE, RMSE e sMAPE (FLOAT64).

`sql/07_key_drivers_and_ai_agg_enhancements.sql (Query 7e + 7f + 7g)`

Funzioni foundation zero-shot per serie storiche integrate direttamente in BigQuery SQL

7e. AI.DETECT_ANOMALIES
Confronto Storico vs Finestra Target

• Prende due TABLE(): lo storico baseline (< 2025-03-10) e la finestra da ispezionare (>= 2025-03-10)

• anomaly_prob_threshold => 0.80 imposta la sensibilità degli intervalli di confidenza

• Restituisce is_anomaly (BOOL), lower_bound, upper_bound e anomaly_probability

7f. AI.FORECAST MULTIVARIATO
Multi-Target + Covariate Storiche

• target_cols => ['defect_count', 'avg_severity'] prevede più metriche simultaneamente

• past_covariate_cols => ['total_reviews'] usa il volume totale recensioni per affinare la stima

• horizon => 7 proietta i 7 giorni successivi con intervalli di confidenza al 90%

7g. AI.EVALUATE
Backtesting Zero-Shot Immediato

• Confronta la finestra di training con i dati reali osservati nella finestra di holdout

• Calcola istantaneamente mean_absolute_error (MAE), root_mean_squared_error (RMSE) e sMAPE

• Permette di validare l'affidabilità di TimesFM 2.5 prima di metterlo in produzione

Costrutto Chiave: TimesFM 2.5: AI.DETECT_ANOMALIES, AI.FORECAST Multivariato & AI.EVALUATE